Guía para principiantes

Flujos de la API y algoritmos de regresión

Curso IFCD107 – Especialista en Inteligencia Artificial

Octubre de 2026

Resumen

Guía paso a paso para entender una API Flask de regresión desde la petición hasta el modelo, las métricas y los archivos resultantes. Explica decisiones de diseño con ejemplos de código comentados para regresión polinómica, búsqueda de hiperparámetros, KNN, Random Forest, SVR, AdaBoost y Gradient Boosting. Los datos de terrazas son sintéticos.

Índice

1. El recorrido de una petición **2**
1.1. Conceptos que aparecen en el código 2

2. Arranque y autenticación **2**

3. Mapa completo de endpoints **3**

4. Validar y preparar datos tabulares **3**

5. Regresión polinómica **4**
5.1. Qué hace y cómo fluye 4

6. Búsqueda polinómica: RandomizedSearchCV **4**

7. KNN: K vecinos más próximos **5**
7.1. K-Fold 5
7.2. Leave-One-Out 5

8. Random Forest **5**

9. SVR: máquinas de vectores soporte para regresión **6**

10. AdaBoostRegressor **6**

11. GradientBoostingRegressor **6**

12. Métricas y archivos de salida **7**

13. Errores HTTP comunes **7**

14. Decisiones de diseño explicadas **7**

15. Resumen visual del flujo **8**

1. El recorrido de una petición

La API separa el trabajo en tres módulos principales: `app.py` recibe HTTP y valida la ruta; `api_functions.py` prepara datos, coordina entrenamiento y guarda resultados; `regression/` construye cada algoritmo. Al final, Flask responde con JSON.

1. El cliente comprueba el servicio con `GET /api/v1/health`.
2. Solicita credenciales a `POST /api/v1/auth/token`.
3. Envía una ruta de entrenamiento con `Authorization: Bearer <token>`.
4. La API valida referencia, parámetros, tamaño y contenido JSON/CSV.
5. La función común separa objetivo y y predictores X, crea partición o folds.
6. Un Pipeline transforma los datos y ajusta el modelo.
7. Se calculan predicciones y métricas, se guardan artefactos y se devuelve JSON.

1.1. Conceptos que aparecen en el código

Predictoras X: columnas de entrada que ayudan a estimar el objetivo.

Objetivo y: número que se desea predecir.

Train/test: entrenamiento enseña al modelo; test se reserva para estimar cómo generaliza.

Pipeline: aplica transformaciones y estimador en orden; evita aprender el preprocesamiento usando test.

Residuo: diferencia entre valor observado y predicho, $y - \hat{y}$.

Hiperparámetro: opción definida antes de entrenar, como grado, número de vecinos o árboles.

2. Arranque y autenticación

Crear un entorno, instalar las dependencias y ejecutar Flask:

```
python -m venv .venv
# Windows PowerShell
.venv\\Scripts\\Activate.ps1
python -m pip install -r requirements.txt
python app.py
```

Configura `API_ADMIN_PASSWORD` y `API_TOKEN_SECRET` usando `.env.example`. No compartas un `.env` con contraseñas reales. La API escucha en http://127.0.0.1:5000.

```
# 1. Health: esta ruta es pblica
GET /api/v1/health

# 2. Pedir token con la contrasea local configurada
POST /api/v1/auth/token
Content-Type: application/json

{"username": "admin", "password": "<clave local>"}

# 3. Usar el token temporal en cada POST de entrenamiento
Authorization: Bearer <access_token>
```

Las rutas de entrenamiento necesitan Bearer válido. Un token ausente, inválido o vencido produce HTTP 401. CSV se envía como `multipart/form-data`, con archivo en el campo `file`; deja que Postman genere el boundary.

3. Mapa completo de endpoints

Ruta	Entrada	Función
`GET /api/v1/health`	Ninguna	Comprobar disponibilidad.
`POST /api/v1/auth/token`	JSON	Emitir Bearer temporal.
`POST /api/v1/regresion/polinomica`	JSON	Ajustar polinomio a puntos x, y.
`POST /api/v1/regresion/polinomica/csv`	CSV	Ajustar con encabezados `x,y`.
`POST /api/v1/regresion/polinomica/randomizedsearch`	JSON o CSV	Buscar hiperparámetros con CV.
`POST /api/v1/regresion/knn/kfold`	CSV	Validar KNN con pliegues K-Fold.
`POST /api/v1/regresion/knn/leave-one-out`	CSV	Validar KNN dejando una fila fuera.
`POST /api/v1/random-forest/terrazas`	JSON	Random Forest con el CSV sintético incluido.
`POST /api/v1/random-forest/csv`	CSV	Random Forest con datos enviados.
`POST /api/v1/regresion/svm/csv`	CSV	Regresión Support Vector (SVR).
`POST /api/v1/regresion/boosting/adaboost/csv`	CSV	AdaBoostRegressor.
`POST /api/v1/regresion/boosting/gradient/csv`	CSV	GradientBoostingRegressor.

4. Validar y preparar datos tabulares

Un CSV tabular tiene una columna objetivo (por defecto `y`) y una o más columnas predictoras. Si existe `expediente_id`, se conserva para identificar una predicción, pero se excluye de X: un número de expediente no debería ser señal predictiva.

```python
import pandas as pd

tabla = pd.read_csv("datos.csv")
y = tabla["y"]  # La cantidad que queremos estimar
X = tabla.drop(columns=["y", "expediente_id"], errors="ignore")
```

La API reconoce columnas numéricas y categóricas. Numéricas: los valores ausentes se rellenan con la mediana. Categóricas: se usa la categoría más frecuente y después One-Hot Encoding. SVR requiere además escalado numérico; KNN estandariza sus predictores numéricos. Bosques y árboles boosting no necesitan escalado.

```python
# Esquema simplificado del preprocesador del proyecto
ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),  # se activa para SVR
    ]), numeric_columns),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), categorical_columns),
])
```

El preprocesador va dentro del Pipeline para que cada fold aprenda medianas, categorías y escalas solo desde su parte de entrenamiento. Así se evita la **filtración de información**.

5. Regresión polinómica

5.1. Qué hace y cómo fluye

Aprende $\hat{y} = a_d x^d + \ldots + a_1 x + a_0$. El grado d controla la curvatura. La ruta JSON valida la lista de pares, la ruta CSV requiere columnas `x,y`. Ambas llaman a la función común: se separa train/test, se calculan coeficientes con train y se calculan métricas sobre test.

```
# Peticin de ejemplo para el endpoint JSON
POST /api/v1/regresion/polinomica
Authorization: Bearer <token>
Content-Type: application/json

{
  "reference": "demo_polinomio",
  "degree": 2,
  "data": [
    {"x": 0, "y": 1}, {"x": 1, "y": 2},
    {"x": 2, "y": 5}, {"x": 3, "y": 10},
    {"x": 4, "y": 17}, {"x": 5, "y": 26}
  ]
}
```

```
# Ncleo del ajuste usado por PolynomialRegressor
coeficientes = np.polyfit(x_train, y_train, deg=2)
prediccion = np.polyval(coeficientes, x_test)
residuos = y_test - prediccion
```

Grados altos pueden seguir muy de cerca train y fallar en ejemplos nuevos. No elegir el grado usando el conjunto test.

6. Búsqueda polinómica: RandomizedSearchCV

La búsqueda reserva primero test. Sobre train prueba combinaciones de grado y diagnósticos mediante folds. Cada ronda aprende en algunos folds y valida en el que queda fuera. Selecciona por RMSE de validación; finalmente prueba el mejor estimador sobre el test reservado.

```
# Esquema legible de la bsqueda del proyecto
pipeline = Pipeline([
    ("polynomial", PolynomialSearchEstimator())
])
search = RandomizedSearchCV(
    pipeline,
    param_distributions={
        "polynomial__degree": [1, 2, 3, 4, 5],
        "polynomial__rcond": [None, 1e-12, 1e-10, 1e-8],
        "polynomial__full": [False, True],
        "polynomial__w": [None],
        "polynomial__cov": [False, "unscaled"],
    },
    n_iter=20,
    cv=5,
    scoring="neg_mean_squared_error",
    random_state=42,
)
search.fit(X_train, y_train)
best_model = search.best_estimator_
prediction = best_model.predict(X_test)
```

`n_iter` es cuántas combinaciones muestrear; `cv` es el número de folds. El código usa error cuadrático medio negativo para ordenar combinaciones y convierte el resultado a RMSE para informarlo. El prefijo `polynomial__` selecciona el paso del Pipeline. Más iteraciones suelen tomar más tiempo.

7. KNN: K vecinos más próximos

KNN predice el promedio de los objetivos de las filas cercanas. Como compara distancias, escala los predictores para que columnas grandes no dominen. En este proyecto KNN requiere tabla numérica, completa y finita.

7.1. K-Fold

Se divide la tabla en k grupos. Cada grupo se predice después de entrenar en los restantes; al final cada fila tiene una predicción fuera de fold (OOF).

```
splitter = KFold(n_splits=5, shuffle=True, random_state=42)
modelo = Pipeline([
    ("scale", StandardScaler()),
    ("knn", KNeighborsRegressor(n_neighbors=5)),
])
predicciones_oof = cross_val_predict(modelo, X, y, cv=splitter)
```

La ruta es `/api/v1/regresion/knn/kfold`; campos típicos: `file`, `target`, `n_neighbors`, `cv`, `random_state`.

7.2. Leave-One-Out

Cada ronda deja una fila fuera y entrena con las demás. Para n filas hace n ajustes; por eso puede ser lento.

```
splitter = LeaveOneOut()
predicciones_loo = cross_val_predict(modelo, X, y, cv=splitter)
```

La ruta es `/api/v1/regresion/knn/leave-one-out`. El CSV de ejemplo se limita a 5.000 filas para mantener un tiempo razonable.

8. Random Forest

Random Forest combina muchos árboles entrenados con variación en muestras y características. Puede aprender relaciones no lineales y no necesita escalar datos. El flujo detecta categorías, imputa valores ausentes, crea variables one-hot, separa test, ajusta y mide.

```
modelo = Pipeline([
    ("preprocess", preprocesador),
    ("random_forest", RandomForestRegressor(
        n_estimators=200,
        max_depth=None,
        random_state=42,
        n_jobs=-1,
    )),
])
modelo.fit(X_train, y_train)
prediction = modelo.predict(X_test)
```

`n_estimators` es el número de árboles; `max_depth` limita niveles. Rutas: `/api/v1/random-forest/terrazas` (datos sintéticos incorporados) y `/api/v1/random-forest/csv`. La importancia describe cuánto contribuye una variable a las particiones del modelo; no prueba causalidad.

9. SVR: máquinas de vectores soporte para regresión

SVR aprende una función tolerando errores menores que ϵ, con penalización controlada por C. El escalado es importante porque magnitudes diferentes cambian distancias y márgenes.

```
modelo = Pipeline([
    ("preprocess", preprocesador_con_escalado),
    ("svm", SVR(
        kernel="rbf", C=1.0, epsilon=0.1,
        gamma="scale", degree=3,
    )),
])
modelo.fit(X_train, y_train)
prediction = modelo.predict(X_test)
```

`kernel` admite `linear`, `poly`, `rbf`, `sigmoid`; `C` penaliza desviaciones, `epsilon` fija el margen tolerado, `gamma` controla influencia en ciertos kernels. `degree` solo importa con kernel polinómico. Endpoint: `/api/v1/regresion/svm/csv`.

10. AdaBoostRegressor

AdaBoost entrena modelos en secuencia y da mayor atención a ejemplos difíciles para los pasos anteriores. El proyecto combina árboles sencillos de profundidad limitada.

```
arbol = DecisionTreeRegressor(max_depth=2, random_state=42)
modelo = Pipeline([
    ("preprocess", preprocesador),
    ("adaboost", AdaBoostRegressor(
        estimator=arbol,
        n_estimators=100,
        learning_rate=0.05,
        loss="linear",
        random_state=42,
    )),
])
modelo.fit(X_train, y_train)
prediction = modelo.predict(X_test)
```

`n_estimators` cuenta pasos; `learning_rate` regula cuánto aporta cada paso; `max_depth` se aplica al árbol base; `loss` elige su función de pérdida. Endpoint: `/api/v1/regresion/boosting/adaboost/csv`.

11. GradientBoostingRegressor

Este método también suma árboles gradualmente. Cada etapa nueva corrige parte del error acumulado hasta ese momento.

```
modelo = Pipeline([
    ("preprocess", preprocesador),
    ("gradient_boosting", GradientBoostingRegressor(
        n_estimators=100,
        learning_rate=0.05,
        max_depth=2,
        loss="squared_error",
        subsample=1.0,
        random_state=42,
    )),
])
modelo.fit(X_train, y_train)
prediction = modelo.predict(X_test)
```

`subsample` es la fracción de filas usada por etapa; una fracción menor puede regularizar. `learning_rate` bajo suele requerir más árboles. Endpoint: `/api/v1/regresion/boosting/gradient/csv`.

12. Métricas y archivos de salida

R^2: comparación con la referencia de predecir siempre la media; en test puede ser negativo.

RMSE: tamaño típico del error, penaliza más los errores grandes.

MAE: media del error absoluto, fácil de explicar en unidades del objetivo.

Una ejecución incluye `reference`, `run_id`, métricas y rutas `artifacts`. Según el algoritmo se guardan CSV de entrada y predicciones, gráfica, modelo pickle y log. Pickle solo se carga si se confía en su origen: un archivo manipulado puede ejecutar código.

13. Errores HTTP comunes

Código	Qué suele significar
400	Falta un campo, encabezado incorrecto, valor no numérico o parámetro fuera de rango.
401	Falta el Bearer o el token venció; pedir uno nuevo.
413	El cuerpo supera 5 MB o el CSV supera 4 MB.
422	La entrada se pudo leer, pero no hay filas suficientes para ese ajuste o validación.
500	Error inesperado; buscar el log asociado a la referencia y ejecución.

14. Decisiones de diseño explicadas

1. **Un módulo por capa:** HTTP, coordinación y algoritmos se mantienen separados para que sea fácil localizar responsabilidades.
2. **Pipeline completo:** las transformaciones se aprenden solo con train o con el train de cada fold; así se evita filtración.
3. **Escalado selectivo:** KNN y SVR comparan distancias/márgenes; árboles eligen cortes y no dependen de la escala.
4. **Semilla reproducible:** permite repetir divisiones y comparar opciones en prácticas.
5. **ID fuera de predictores:** evita que el modelo memorice claves particulares.
6. **Test reservado:** la búsqueda ocurre en train; test se consulta al final para una estimación menos sesgada.
7. **Archivos por ejecución:** referencia y run-id evitan sobrescribir salidas anteriores.
8. **Datos de demostración:** los expedientes de terrazas son sintéticos y no permiten inferencias municipales reales.

15. Resumen visual del flujo

```
Postman
  -> health
  -> obtener token
  -> elegir endpoint + JSON/CSV + Bearer
  -> app.py autentica y valida
  -> api_functions.py prepara train/test o folds
  -> regression/<modelo>.py construye y ajusta
  -> calcular predicciones y mtricas
  -> guardar artefactos bajo runs/<reference>/
  -> responder JSON con run_id y rutas
```